MATERIAL DATA MANAGEMENT SYSTEM

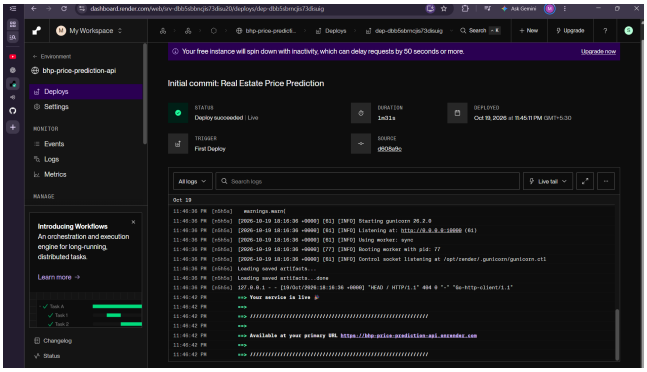
Product Report

Product ID: CASE-01
Product Name: Case 1 - Full 3 Microstructure
Batch: batch-03 week 3
Date: 05-11-2026

Generated on 03-11-2026 10:00

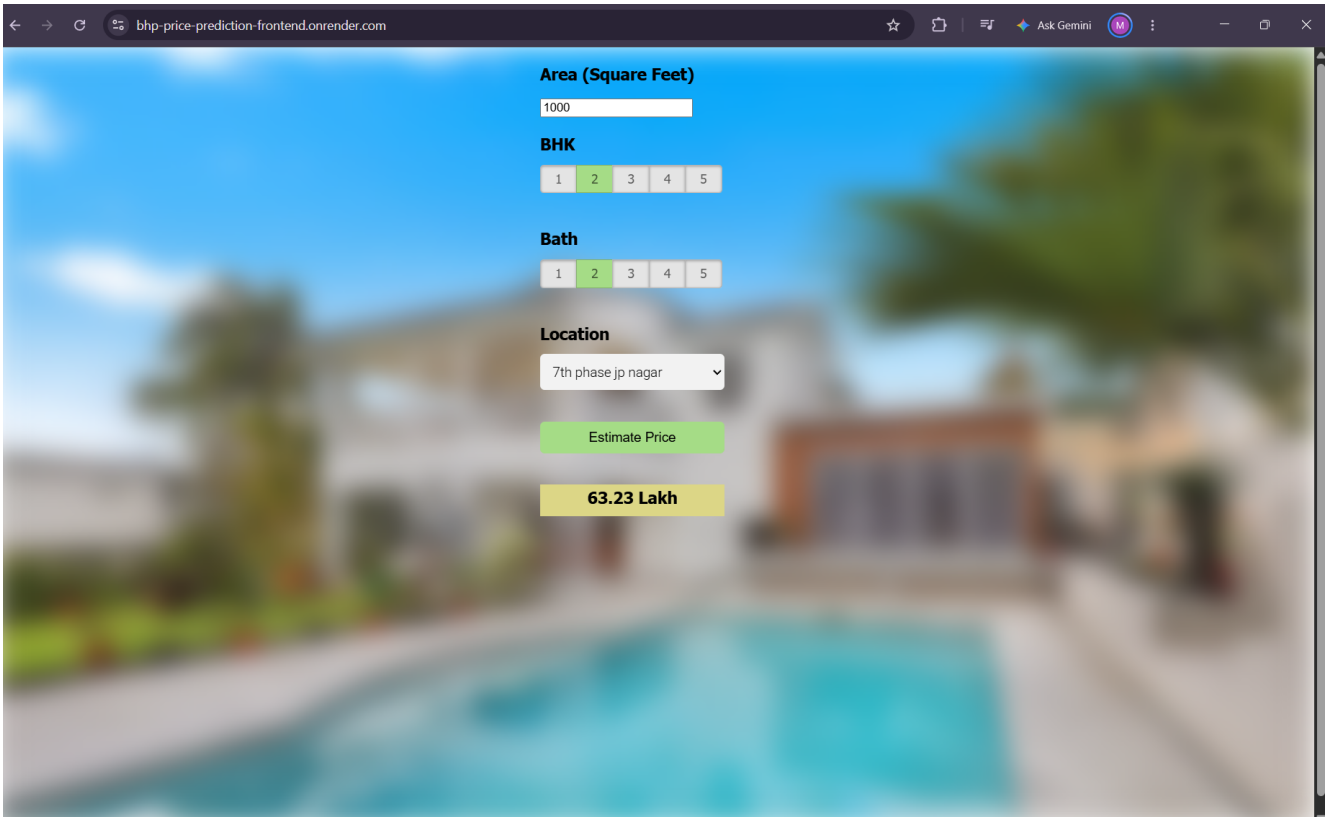
Microstructure Images

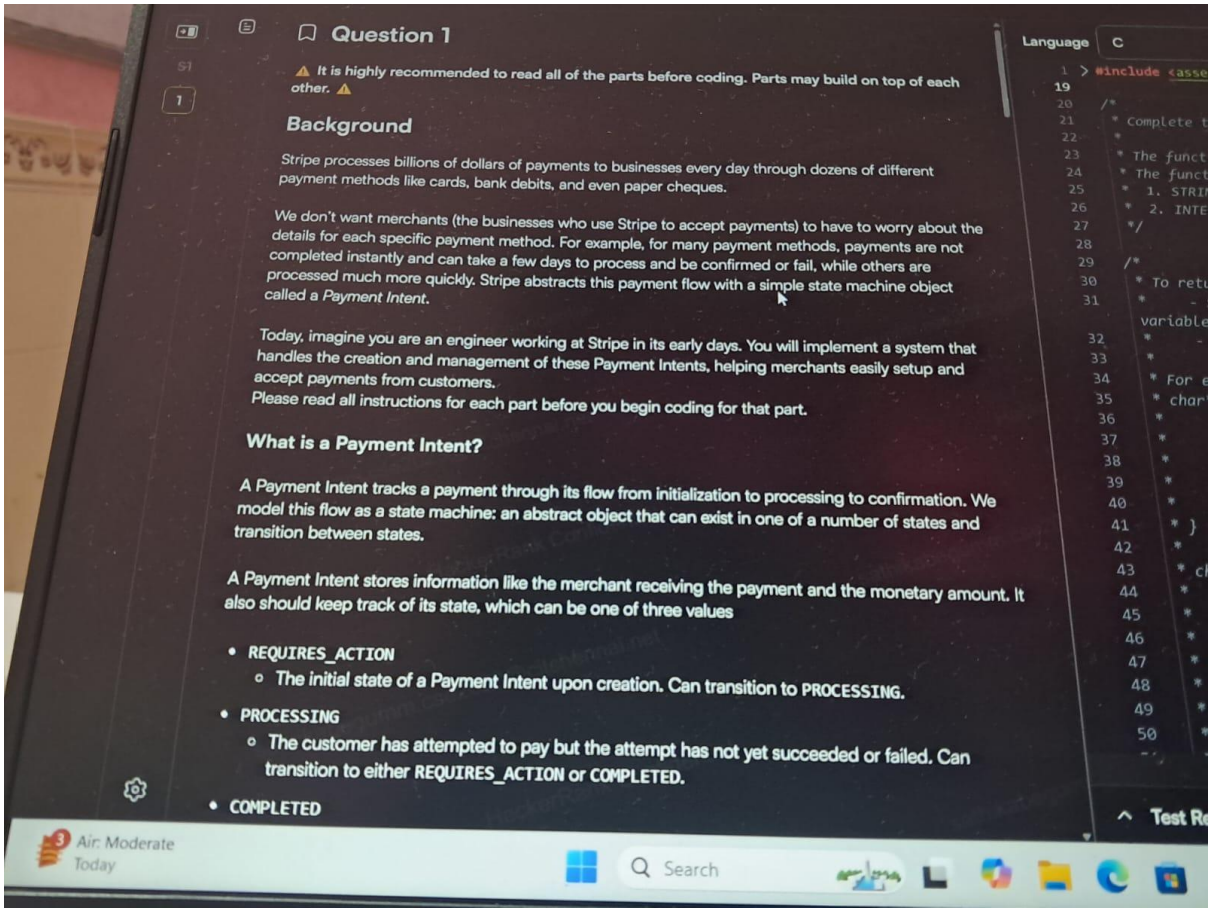


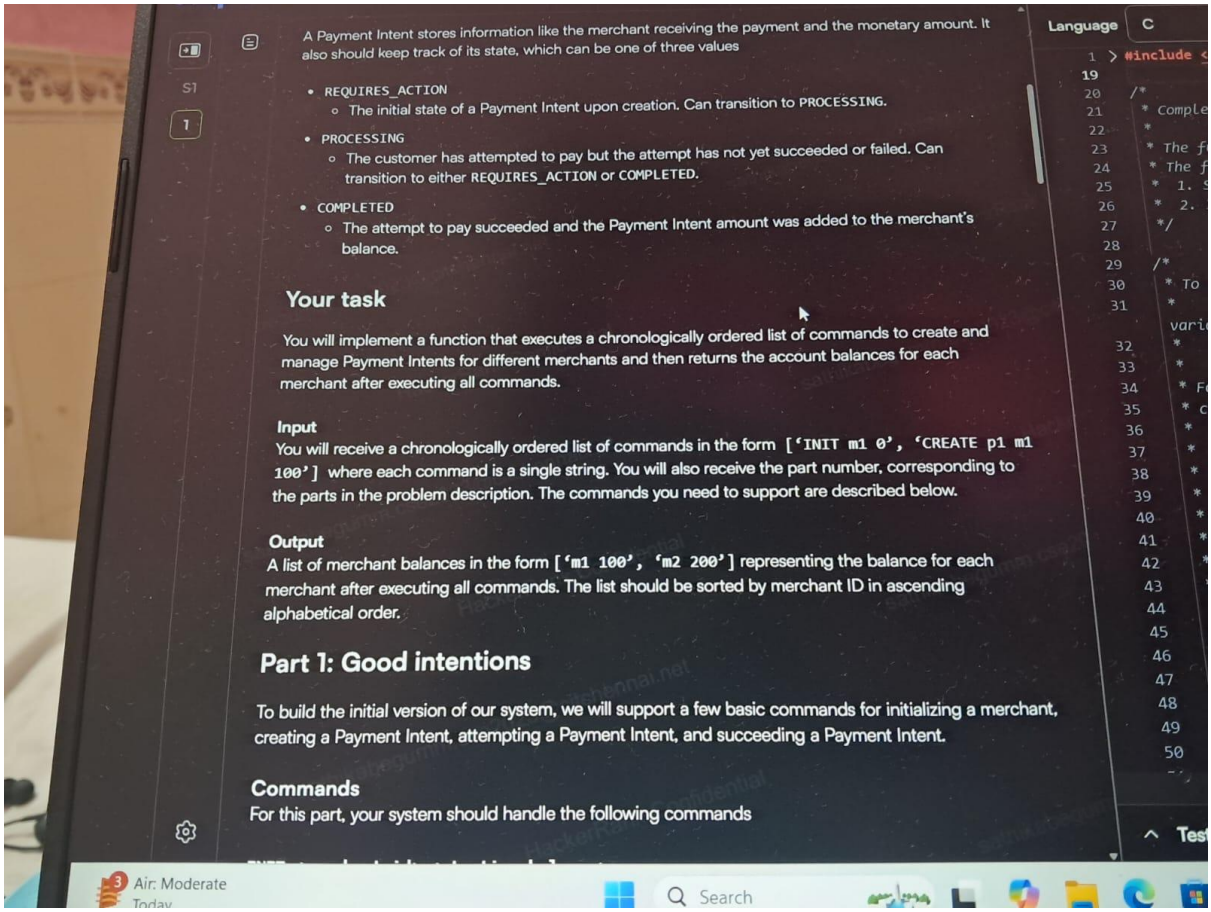


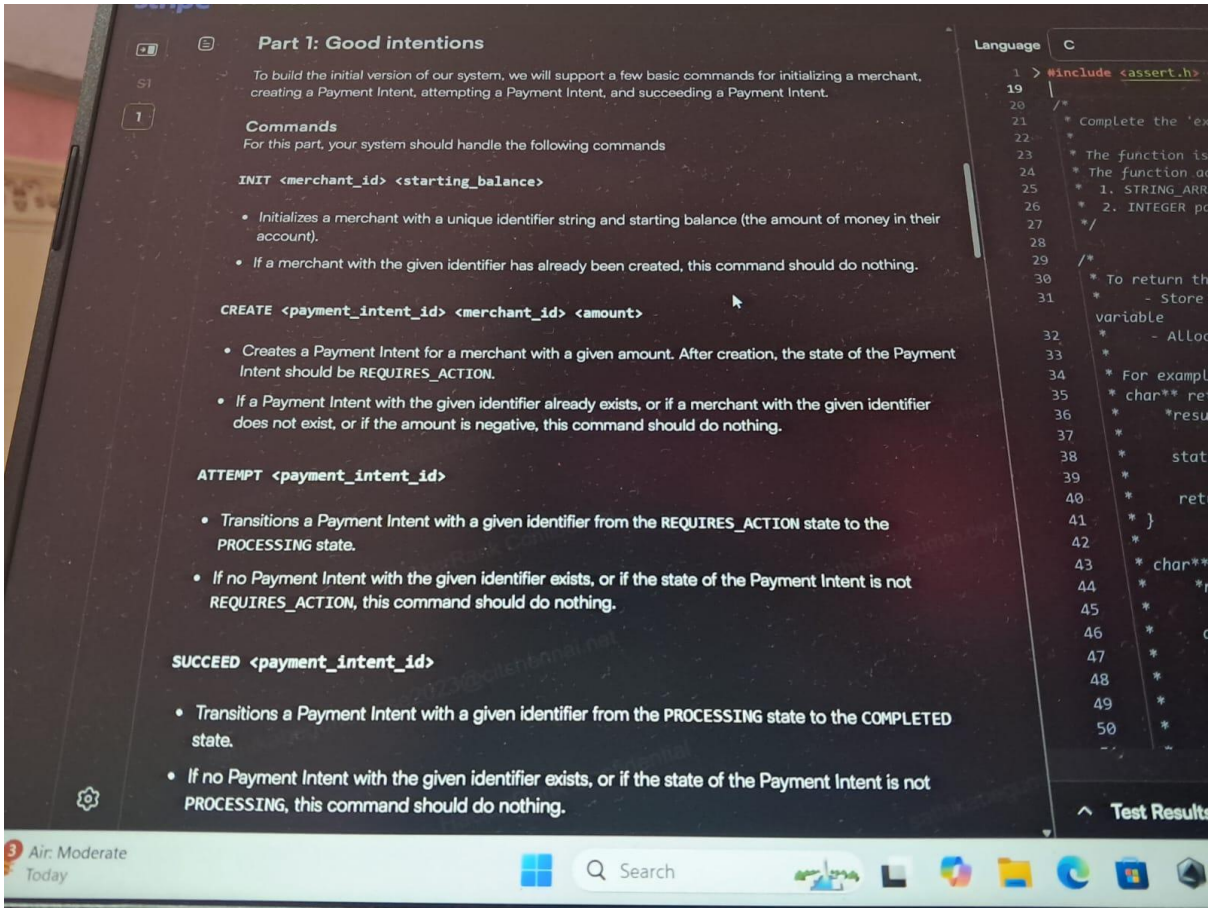


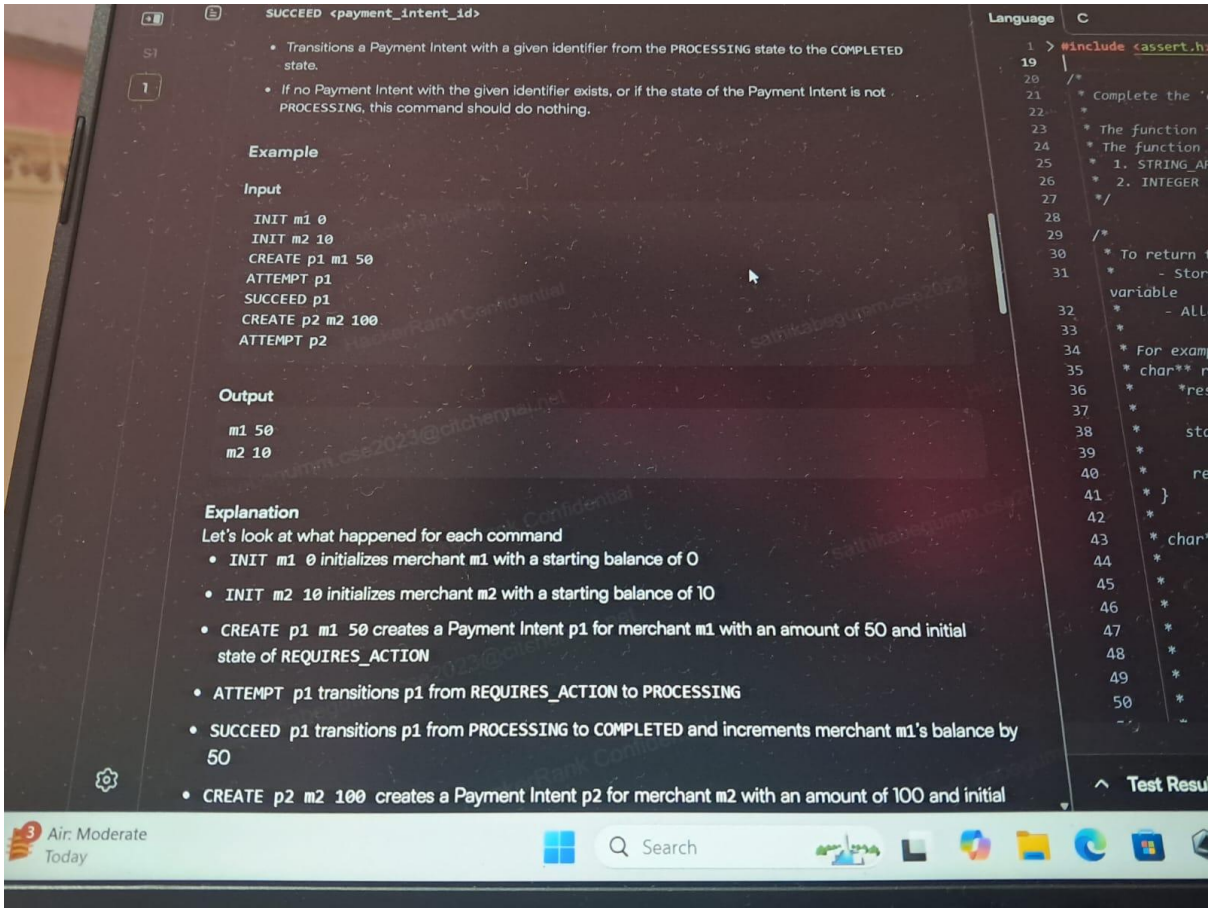
Graph

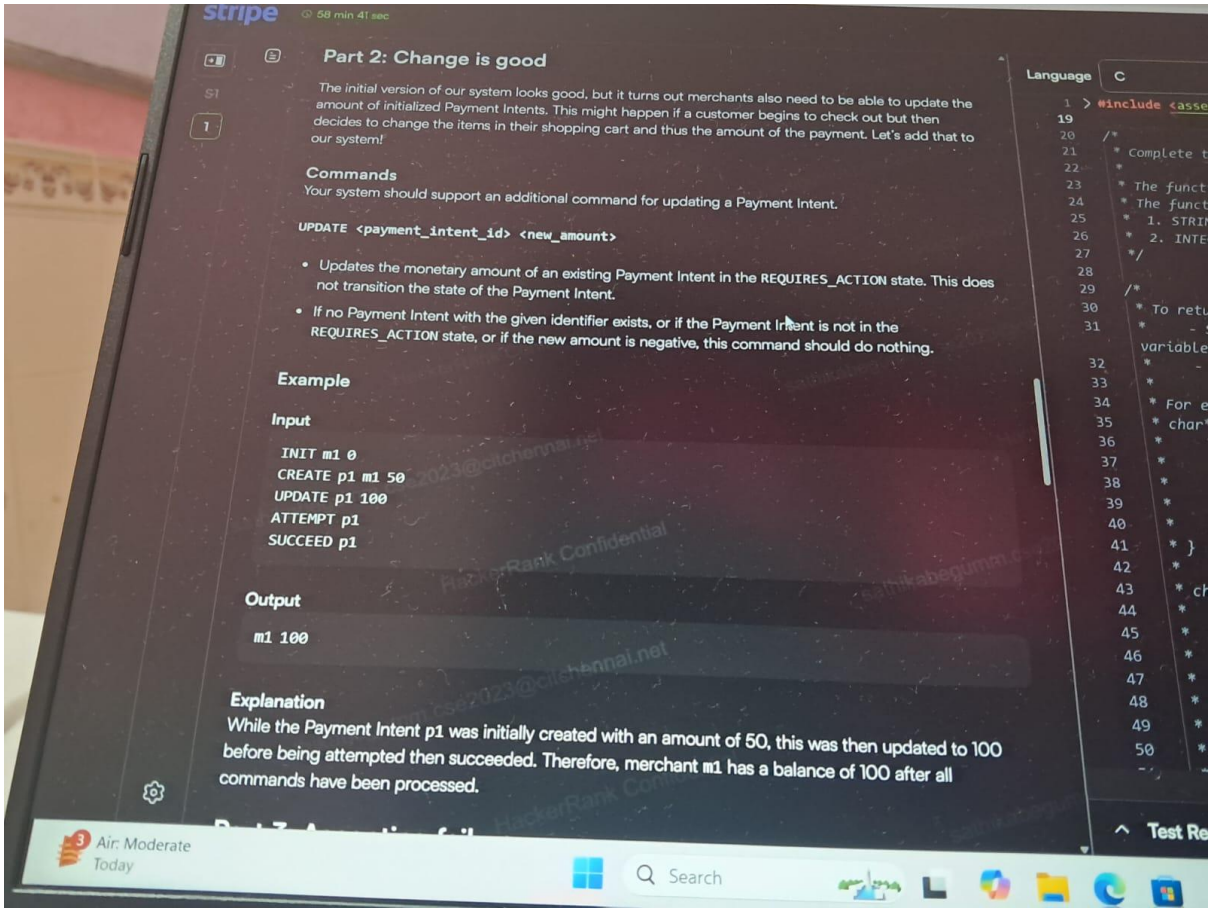

bhp-price-prediction-frontend.onrender.com
Ask Gemini
Area (Square Feet)
1000
BHK
1 2 3 4 5
Bath
1 2 3 4 5
Location
7th phase jp nagar
Estimate Price
63.23 Lakh

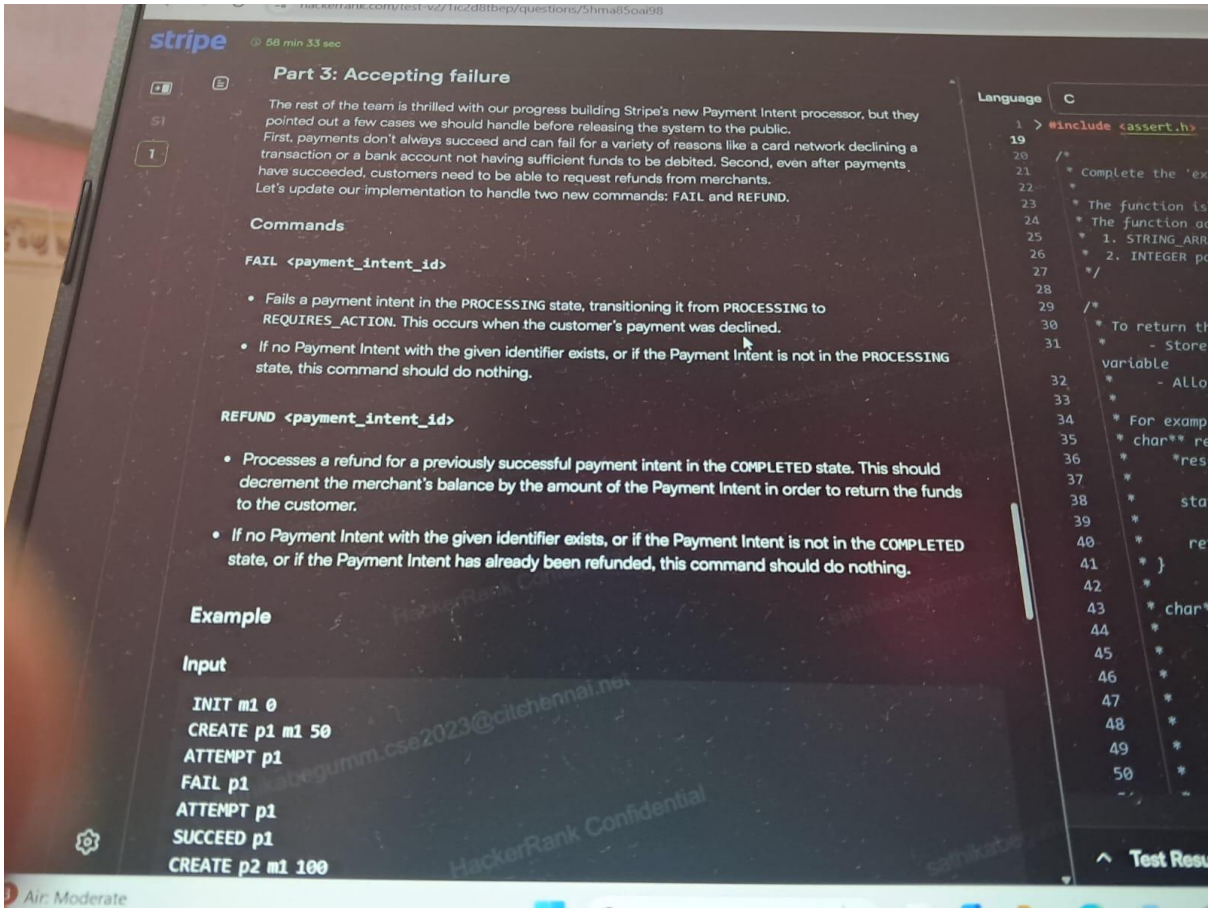

Question 1
⚠ It is highly recommended to read all of the parts before coding. Parts may build on top of each other. ⚠
Background
Stripe processes billions of dollars of payments to businesses every day through dozens of different payment methods like cards, bank debits, and even paper cheques.
We don't want merchants (the businesses who use Stripe to accept payments) to have to worry about the details for each specific payment method. For example, for many payment methods, payments are not completed instantly and can take a few days to process and be confirmed or fail, while others are processed much more quickly. Stripe abstracts this payment flow with a simple state machine object called a Payment Intent.
Today, imagine you are an engineer working at Stripe in its early days. You will implement a system that handles the creation and management of these Payment Intents, helping merchants easily setup and accept payments from customers.
Please read all instructions for each part before you begin coding for that part.
What is a Payment Intent?
A Payment Intent tracks a payment through its flow from initialization to processing to confirmation. We model this flow as a state machine: an abstract object that can exist in one of a number of states and transition between states.
A Payment Intent stores information like the merchant receiving the payment and the monetary amount. It also should keep track of its state, which can be one of three values
REQUIRES_ACTION
The initial state of a Payment Intent upon creation. Can transition to PROCESSING.
PROCESSING
The customer has attempted to pay but the attempt has not yet succeeded or failed. Can transition to either REQUIRES_ACTION or COMPLETED.
COMPLETED
Language
C
Air: Moderate
Today
Search

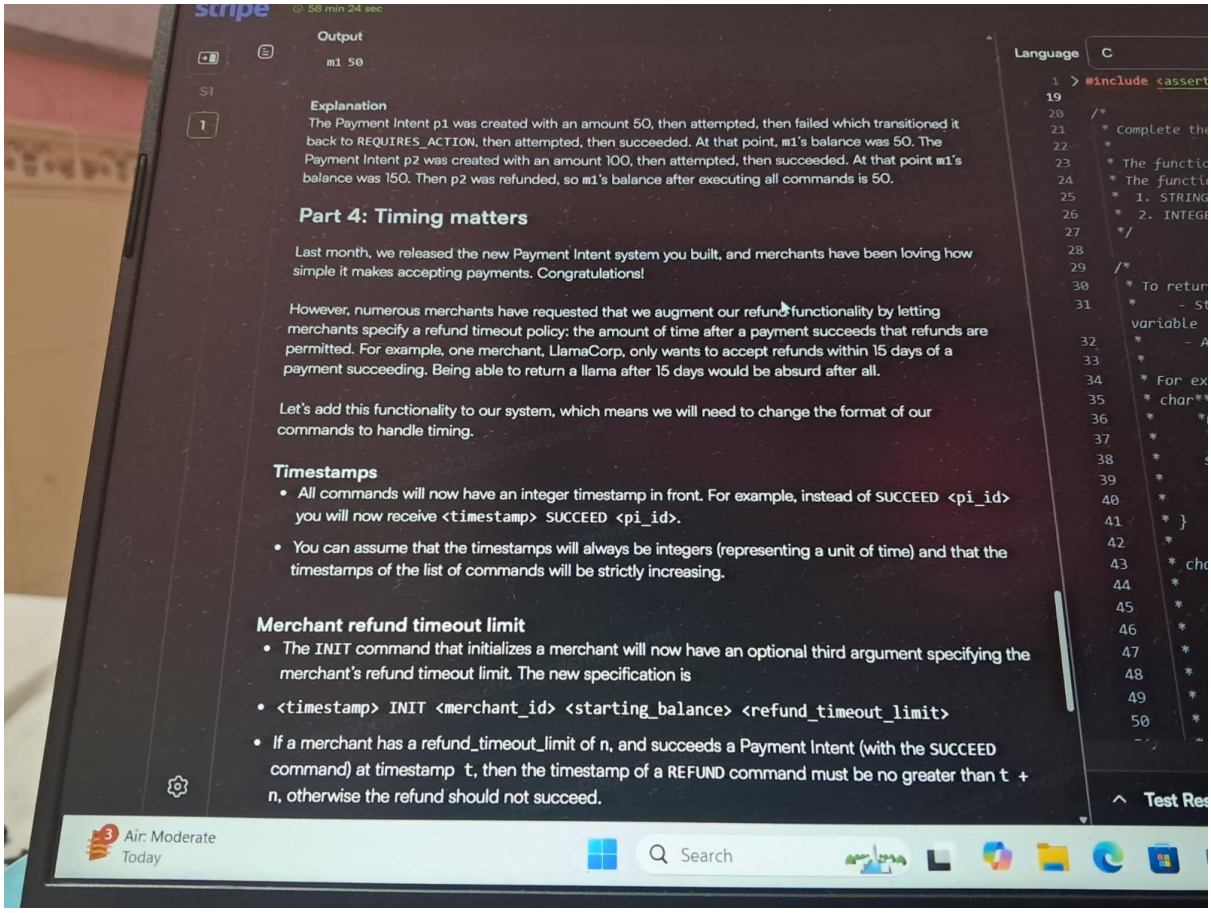

A Payment Intent stores information like the merchant receiving the payment and the monetary amount. It also should keep track of its state, which can be one of three values
REQUIRES_ACTION
The initial state of a Payment Intent upon creation. Can transition to PROCESSING.
PROCESSING
The customer has attempted to pay but the attempt has not yet succeeded or failed. Can transition to either REQUIRES_ACTION or COMPLETED.
COMPLETED
The attempt to pay succeeded and the Payment Intent amount was added to the merchant's balance.
Your task
You will implement a function that executes a chronologically ordered list of commands to create and manage Payment Intents for different merchants and then returns the account balances for each merchant after executing all commands.
Input
You will receive a chronologically ordered list of commands in the form ['INIT m1 0', 'CREATE p1 m1 100'] where each command is a single string. You will also receive the part number, corresponding to the parts in the problem description. The commands you need to support are described below.
Output
A list of merchant balances in the form ['m1 100', 'm2 200'] representing the balance for each merchant after executing all commands. The list should be sorted by merchant ID in ascending alphabetical order.
Part 1: Good intentions
To build the initial version of our system, we will support a few basic commands for initializing a merchant, creating a Payment Intent, attempting a Payment Intent, and succeeding a Payment Intent.
Commands
For this part, your system should handle the following commands
Language
C
Air: Moderate
Today
Search

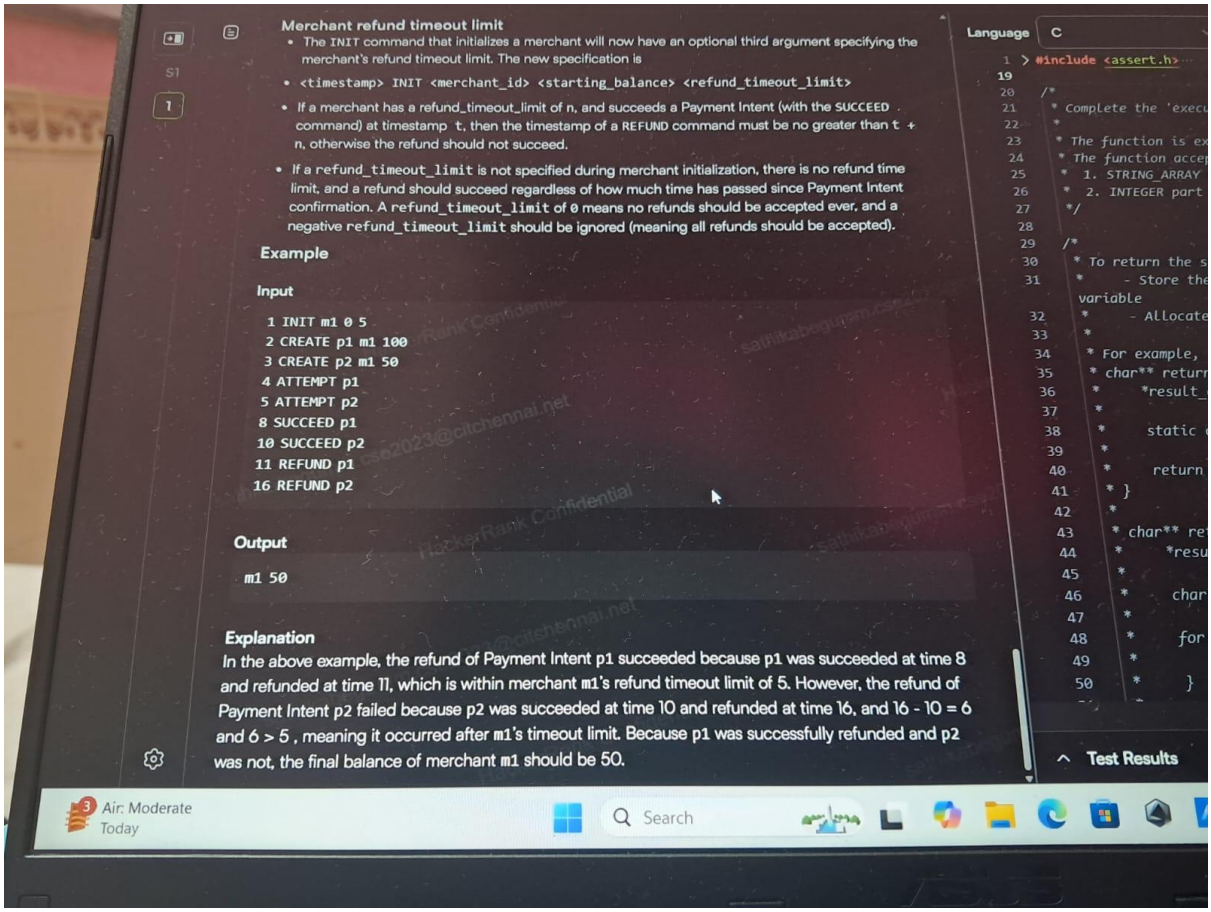

Part 1: Good intentions
To build the initial version of our system, we will support a few basic commands for initializing a merchant, creating a Payment Intent, attempting a Payment Intent, and succeeding a Payment Intent.
Commands
For this part, your system should handle the following commands
INIT <merchant_id> <starting_balance>
Initializes a merchant with a unique identifier string and starting balance (the amount of money in their account).
If a merchant with the given identifier has already been created, this command should do nothing.
CREATE <payment_intent_id> <merchant_id> <amount>
Creates a Payment Intent for a merchant with a given amount. After creation, the state of the Payment Intent should be REQUIRES_ACTION.
If a Payment Intent with the given identifier already exists, or if a merchant with the given identifier does not exist, or if the amount is negative, this command should do nothing.
ATTEMPT <payment_intent_id>
Transitions a Payment Intent with a given identifier from the REQUIRES_ACTION state to the PROCESSING state.
If no Payment Intent with the given identifier exists, or if the state of the Payment Intent is not REQUIRES_ACTION, this command should do nothing.
SUCCEED <payment_intent_id>
Transitions a Payment Intent with a given identifier from the PROCESSING state to the COMPLETED state.
If no Payment Intent with the given identifier exists, or if the state of the Payment Intent is not PROCESSING, this command should do nothing.
Language
C
#include <assert.h>
Test Results
Air: Moderate
Today
Search


SUCCEED <payment_intent_id>
Transitions a Payment Intent with a given identifier from the PROCESSING state to the COMPLETED state.
If no Payment Intent with the given identifier exists, or if the state of the Payment Intent is not PROCESSING, this command should do nothing.
Example
Input
INIT m1 0
INIT m2 10
CREATE p1 m1 50
ATTEMPT p1
SUCCEED p1
CREATE p2 m2 100
ATTEMPT p2
Output
m1 50
m2 10
Explanation
Let's look at what happened for each command
INIT m1 0 initializes merchant m1 with a starting balance of 0
INIT m2 10 initializes merchant m2 with a starting balance of 10
CREATE p1 m1 50 creates a Payment Intent p1 for merchant m1 with an amount of 50 and initial state of REQUIRES_ACTION
ATTEMPT p1 transitions p1 from REQUIRES_ACTION to PROCESSING
SUCCEED p1 transitions p1 from PROCESSING to COMPLETED and increments merchant m1's balance by 50
CREATE p2 m2 100 creates a Payment Intent p2 for merchant m2 with an amount of 100 and initial
Language
C
#include <assert.h
Test Resu
Air: Moderate
Today
Search


stripe
58 min 41 sec
Part 2: Change is good
The initial version of our system looks good, but it turns out merchants also need to be able to update the amount of initialized Payment Intents. This might happen if a customer begins to check out but then decides to change the items in their shopping cart and thus the amount of the payment. Let's add that to our system!
Commands
Your system should support an additional command for updating a Payment Intent.
UPDATE <payment_intent_id> <new_amount>
Updates the monetary amount of an existing Payment Intent in the REQUIRES_ACTION state. This does not transition the state of the Payment Intent.
If no Payment Intent with the given identifier exists, or if the Payment Intent is not in the REQUIRES_ACTION state, or if the new amount is negative, this command should do nothing.
Example
Input
INIT m1 0
CREATE p1 m1 50
UPDATE p1 100
ATTEMPT p1
SUCCEED p1
Output
m1 100
Explanation
While the Payment Intent p1 was initially created with an amount of 50, this was then updated to 100 before being attempted then succeeded. Therefore, merchant m1 has a balance of 100 after all commands have been processed.
Language
C
Air: Moderate
Today
Search


stripe
58 min 33 sec
Part 3: Accepting failure
The rest of the team is thrilled with our progress building Stripe's new Payment Intent processor, but they pointed out a few cases we should handle before releasing the system to the public.
First, payments don't always succeed and can fail for a variety of reasons like a card network declining a transaction or a bank account not having sufficient funds to be debited. Second, even after payments have succeeded, customers need to be able to request refunds from merchants.
Let's update our implementation to handle two new commands: FAIL and REFUND.
Commands
FAIL <payment_intent_id>
Fails a payment intent in the PROCESSING state, transitioning it from PROCESSING to REQUIRES_ACTION. This occurs when the customer's payment was declined.
If no Payment Intent with the given identifier exists, or if the Payment Intent is not in the PROCESSING state, this command should do nothing.
REFUND <payment_intent_id>
Processes a refund for a previously successful payment intent in the COMPLETED state. This should decrement the merchant's balance by the amount of the Payment Intent in order to return the funds to the customer.
If no Payment Intent with the given identifier exists, or if the Payment Intent is not in the COMPLETED state, or if the Payment Intent has already been refunded, this command should do nothing.
Example
Input
INIT m1 0
CREATE p1 m1 50
ATTEMPT p1
FAIL p1
ATTEMPT p1
SUCCEED p1
CREATE p2 m1 100
Language
C
Air: Moderate


stripe
58 min 24 sec
Output
m1 50
Explanation
The Payment Intent p1 was created with an amount 50, then attempted, then failed which transitioned it back to REQUIRES_ACTION, then attempted, then succeeded. At that point, m1's balance was 50. The Payment Intent p2 was created with an amount 100, then attempted, then succeeded. At that point m1's balance was 150. Then p2 was refunded, so m1's balance after executing all commands is 50.
Part 4: Timing matters
Last month, we released the new Payment Intent system you built, and merchants have been loving how simple it makes accepting payments. Congratulations!
However, numerous merchants have requested that we augment our refund functionality by letting merchants specify a refund timeout policy: the amount of time after a payment succeeds that refunds are permitted. For example, one merchant, LlamaCorp, only wants to accept refunds within 15 days of a payment succeeding. Being able to return a llama after 15 days would be absurd after all.
Let's add this functionality to our system, which means we will need to change the format of our commands to handle timing.
Timestamps
All commands will now have an integer timestamp in front. For example, instead of SUCCEED <pi_id> you will now receive <timestamp> SUCCEED <pi_id>.
You can assume that the timestamps will always be integers (representing a unit of time) and that the timestamps of the list of commands will be strictly increasing.
Merchant refund timeout limit
The INIT command that initializes a merchant will now have an optional third argument specifying the merchant's refund timeout limit. The new specification is
<timestamp> INIT <merchant_id> <starting_balance> <refund_timeout_limit>
If a merchant has a refund_timeout_limit of n, and succeeds a Payment Intent (with the SUCCEED command) at timestamp t, then the timestamp of a REFUND command must be no greater than t + n, otherwise the refund should not succeed.
Language
C
Test Res
Air: Moderate
Today
Search


Merchant refund timeout limit
The INIT command that initializes a merchant will now have an optional third argument specifying the merchant's refund timeout limit. The new specification is
<timestamp> INIT <merchant_id> <starting_balance> <refund_timeout_limit>
If a merchant has a refund_timeout_limit of n, and succeeds a Payment Intent (with the SUCCEED command) at timestamp t, then the timestamp of a REFUND command must be no greater than t + n, otherwise the refund should not succeed.
If a refund_timeout_limit is not specified during merchant initialization, there is no refund time limit, and a refund should succeed regardless of how much time has passed since Payment Intent confirmation. A refund_timeout_limit of 0 means no refunds should be accepted ever, and a negative refund_timeout_limit should be ignored (meaning all refunds should be accepted).
Example
Input
1 INIT m1 0 5
2 CREATE p1 m1 100
3 CREATE p2 m1 50
4 ATTEMPT p1
5 ATTEMPT p2
8 SUCCEED p1
10 SUCCEED p2
11 REFUND p1
16 REFUND p2
Output
m1 50
Explanation
In the above example, the refund of Payment Intent p1 succeeded because p1 was succeeded at time 8 and refunded at time 11, which is within merchant m1's refund timeout limit of 5. However, the refund of Payment Intent p2 failed because p2 was succeeded at time 10 and refunded at time 16, and 16 - 10 = 6 and 6 > 5 , meaning it occurred after m1's timeout limit. Because p1 was successfully refunded and p2 was not, the final balance of merchant m1 should be 50.
Language
C
Test Results
Air: Moderate
Today
Search

WhatsApp Image

